```
For every custome, calculate:

Customer ID
Customer name
Number of orders placed
Total amount paid through successful payments only

Display only customers whose successful payment amount is
greater than ₹50,000.

Sort customers from highest to lowest successful payment amount.

select *
from `e1.customers`
limit 5


select *
from `e1.orders`
limit 5


select *
from `e1.payments`
limit 5


c.customerid -> o.customerid
p.orderid -> o.orderid
```

```sql
select
  c.customerID,
  c.name,
  count(distinct o.orderID) as no_of_orders,
  sum(p.amount) as total_paid
from `e1.customers` as c
inner join `e1.orders` as o
  on c.customerID = o.customerID
inner join `e1.payments` as p
  on p.orderID = o.orderID
where p.status = "Success"
group by c.customerID, c.name
having total_paid > 50000
order by total_paid desc
```

Find the top 10 products by total sales value.

Display:

ProductID --> product
ProductName --> product
CategoryName --> category
SupplierName --> supplier
Number of times the product appeared in order items --> order_items
Total quantity sold--> order_items
Total sales value --> order_items
Average selling price --> order_items

```
Only include products where the total sales value is greater
than ₹1,00,000.


-- product, category, supplier, order_items



select
  p.ProductID,
  p.ProductName,
  c.CategoryName,
  s.SupplierName,
  count(oi.OrderID) as no_of_records,
  sum(oi.Quantity) as total_qty,
  sum(oi.Total) as total_sales,
  avg(oi.sellingprice) as avg_selling_price
from `e1.products` as p
inner join `e1.categories` as c
  on p.CategoryID = c.CategoryID
inner join `e1.suppliers` as s
  on p.SupplierID = s.SupplierID
inner join `e1.order_items` as oi
  on p.ProductID = oi.ProductID
group by
  p.ProductID,
  p.ProductName,
  c.CategoryName,
  s.SupplierName
```

```
having total_sales > 100000
order by total_sales desc
limit 10



Consider only products having an MRP greater than ₹20,000.

For every category, calculate:

CategoryID --> category
CategoryName --> category
Number of qualifying products --> products
Average MRP of qualifying products --> products
Maximum MRP --> products
Minimum MRP --> products

Display only categories having more than 2 qualifying products.

Sort by average MRP from highest to lowest.




select
  c.CategoryID,
  c.CategoryName,
  count(p.ProductID) as no_of_products,
  round(avg(p.MRP),2) as avg_mrp,
  max(p.MRP) as max_mrp,
```

```
  min(p.MRP) as min_mrp
from `e1.categories` as c
inner join `e1.products` as p
  on c.CategoryID = p.CategoryID
where p.MRP > 2000
group by
  c.CategoryID,
  c.CategoryName
having  no_of_products > 2
order by avg_mrp desc


Find all customers who have never placed an order. --> order
table -> null

Display:

CustomerID --> customer
Customer name --> customer
City --> customer

Sort the result alphabetically by customer name.


select
  c.CustomerID,
  c.Name,
  c.City,
  o.OrderID
from `e1.customers` as c
```

```
left join `e1.orders` as o
on c.CustomerID = o.CustomerID
where o.OrderID is null
order by c.Name



For every supplier, calculate:

SupplierID --> supplier
SupplierName --> supplier
Number of products supplied --> product
Average MRP of their products --> product
Total sales generated by their products --> order_items
Total quantity sold --> order_items

Include suppliers even if their products have never been sold.
--> priority to suppliers

For suppliers with no sales, display the sales and quantity as
0.

Sort suppliers by total sales from highest to lowest.



select
  s.SupplierID,
  s.SupplierName,
  count(distinct p.ProductID) as prd_cnt,
  avg(p.MRP) as avg_mrp,
```

```
  coalesce(sum(oi.Total),0) as total_sales,
  coalesce(sum(oi.Quantity),0) as total_qty
from `e1.suppliers` as s
left join `e1.products` as p
  on s.SupplierID = p.SupplierID
left join `e1.order_items` as oi
  on p.ProductID = oi.ProductID
group by
  s.SupplierID,
  s.SupplierName
order by total_sales desc
```

For every warehouse, calculate:

WarehouseID
Warehouse name
Number of orders handled
Total quantity of products sold
Total sales value

Include warehouses even if they have no orders.

Display only warehouses where the total sales value is greater than ₹2,00,000.

Sort from highest to lowest total sales.

Calculate the total sales value generated by every customer.

Then display only those customers whose total spending is greater than the average customer spending.

Display:

CustomerID
Customer name
Total spending

Sort from highest spending to lowest.

For every product, calculate its total sales value.

Display only products whose total sales value is greater than the average total sales value of products belonging to the same category.

Display:

ProductID
ProductName

CategoryName
TotalSales

Sort by total sales from highest to lowest.

For every order, calculate the total order value by adding the Total column from order_items.

Then classify each order as:

High Value → Total order value ≥ ₹50,000
Medium Value → Total order value between ₹20,000 and ₹49,999
Low Value → Total order value < ₹20,000

Display:

OrderID
OrderDate
CustomerName
TotalOrderValue
OrderCategory

Show the highest-value orders first.

Create a category-level sales report containing:

CategoryID
CategoryName
Number of products in the category
Number of different products sold
Total quantity sold
Total sales value
Average selling price

Only consider order-item records where:

SellingPrice > ₹10,000

Display only categories where:

Total sales value > ₹5,00,000

Sort categories from highest to lowest total sales.